Respuesta a Pregunta 3
Monitoreo de Fauna Silvestre con Cámaras Trampa (YOLOv8)

Grupo 3

Roxana Vargas – olinda.vargas.c@uni.pe

Nicolas Nieto Alvarado – nicolas.nieto.a@uni.pe

Sebastián Villanueva – elias.villanueva.p@uni.pe

Victor Tello – victor.tello.a@uni.pe

Universidad Nacional de Ingeniería (UNI) - Maestría en Inteligencia Artificial

Abstract

Las cámaras trampa generan más fotografías de las que un equipo humano puede revisar, y la mayoría no contiene ningún animal. Entrenamos YOLOv8s sobre Caltech Camera Traps (243,100 fotos, 140 cámaras) con todas las fotos vacías como fondo explícito y particiones separadas por cámara. En 39 cámaras nunca vistas el modelo alcanza mAP@0.5 = 0.481 y mAP@0.5:0.95 = 0.310, con AP entre 0.87 (zarigüeya) y 0 (lagartija, 32 cajas de entrenamiento). Con la partición oficial ECCV 2018, el mAP@0.5 cae de 0.746 en cámaras vistas a 0.420 en cámaras nuevas, y el modelo genera falsas alarmas en el 38 % de las fotos vacías de cámaras nuevas frente al 16 % de las conocidas; EigenCAM muestra que la activación dominante del backbone describe la escena y no al animal. La oclusión es el factor que más degrada la detección (recall 0.63 con oclusión baja frente a 0.16 con oclusión alta). Un módulo de reglas convierte las detecciones de una cámara no vista en un reporte de actividad por especie y periodo del día que recupera los patrones nocturnos con precisión por evento de 0.76.

1. Motivación

Las cámaras trampa se activan por movimiento y funcionan sin operador, por lo que son la herramienta más usada para monitorear fauna en áreas protegidas, incluidas las de la Amazonía y los Andes peruanos. Su problema es el volumen: una sola cámara produce miles de fotos al año y la mayoría son activaciones falsas por viento, vegetación o cambios de luz. Un detector automático puede descartar las vacías e identificar la especie en las demás, pero solo es útil si funciona en cámaras nuevas, donde el fondo, la iluminación y las especies cambian.

Este trabajo entrena y evalúa un detector YOLOv8 sobre Caltech Camera Traps (CCT) atendiendo a las cuatro dificultades del enunciado: el desbalance extremo (fotos vacías y especies raras), la oclusión por vegetación, los animales pequeños o lejanos y la generalización a cámaras no vistas. Además, implementa un módulo de reglas que agrega las detecciones en un reporte de actividad como el que usaría un equipo de manejo de fauna.

2. Trabajos Relacionados

Norouzzadeh et al. [2] mostraron que las redes convolucionales pueden identificar, contar y describir animales en millones de fotos de Snapshot Serengeti con precisión cercana a la de

voluntarios, y Tabak et al. [4] entrenaron clasificadores de especies para fotos de Norteamérica, advirtiendo que el desempeño baja fuera de las ubicaciones de entrenamiento. Schneider et al. [5] compararon detectores de objetos (Faster R-CNN y YOLO) en datos de cámaras trampa y mostraron que localizar al animal antes de clasificarlo mejora los resultados.

Beery et al. [1] publicaron CCT y su subconjunto CCT-20 con la partición cis/trans que usamos en E3, y documentaron que los modelos memorizan el fondo de cada cámara: el error crece mucho en ubicaciones nuevas. Para enfrentar el volumen de fotos vacías, Beery et al. [3] propusieron MegaDetector, un detector genérico de animales, personas y vehículos que filtra las vacías antes de clasificar. Nuestro detector sigue la familia YOLO [6] en la implementación de Ultralytics [7], con anotaciones en formato COCO [10]. Para inspeccionar en qué se fija el modelo usamos EigenCAM [8], una variante de Grad-CAM [9] que no necesita gradientes ni una clase objetivo y por eso se adapta mejor a detectores de una etapa.

3. Metodología

3.1. Datos y particiones

Usamos las 243,100 fotos de CCT en resolución original (metadatos de 2021, cajas de 2020). En esta versión el 51.7 % de las fotos están vacías, no el ~70 % que se reporta para el dataset; lo verificamos contando las etiquetas de cuatro formas. Encontramos además problemas de calidad que condicionan las decisiones: (i) 2,555 de las 2,557 cajas de *car* cubren la imagen completa, así que excluimos esa clase; (ii) las etiquetas se asignan por secuencia de disparo, de modo que hay fotos etiquetadas con un animal que ya salió del cuadro; (iii) las fotos tomadas entre las 0:00 y las 0:59 aparecen registradas a las 12:xx (error AM/PM), y (iv) algunas cámaras usan flash blanco de noche, por lo que una foto a color no implica que sea de día. Además, el 52 % de las fotos con animal son nocturnas, casi todas en infrarrojo y sin color (Figura 1).

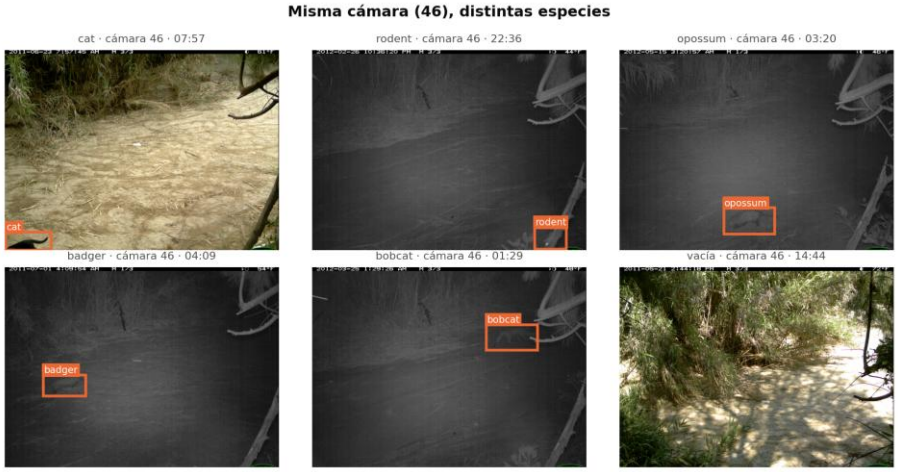


Figura 1: Seis fotos de la cámara 46 (la del demo de E5): el fondo se repite en todas, la mayoría de los animales ocupan una fracción pequeña de la imagen y la cámara alterna color de día e infrarrojo de noche.

Excluimos las clases casi sin cajas (pig, cow, bat, insect) y las 49,801 fotos con animal pero sin caja, porque YOLO las aprendería como fondo. Quedan 16 especies y 184,945 fotos (59,200 con caja y 125,745 vacías). Entre ellas el desbalance es de 433 veces: la zarigüeya tiene 12,134 cajas y el tejón 28. Todas las particiones se separan por cámara para que el modelo no pueda reconocer en test un fondo visto en entrenamiento (Tabla 1). La partición *full* sale de la división de LILA en 100 cámaras de entrenamiento y 40 de validación: de las primeras tomamos 76 para entrenar y 19 para seleccionar la época; las 39 restantes forman el test. La partición *eccv* es la oficial de ECCV 2018 [1], que el enunciado exige para E3. De su conjunto de validación usamos eccv_cis_val (cámaras vistas) para elegir la época del modelo E3; no usamos eccv_trans_val para no elegir el modelo mirando cámaras nuevas, y reportamos la brecha en los conjuntos de test oficiales, más grandes.

Tabla 1: Particiones usadas (16 especies). Las particiones separan cámaras completas.

Partición	Cámaras	Imágenes	Vacías	Cajas	Uso
full_train	76	113,642	94,856	19,861	Entrenar el modelo principal
full_val	19	12,580	8,867	3,884	Elegir la mejor época
full_test	39	43,292	19,892	24,662	E2, E4, E5 (cámaras no vistas)
eccv_train	10	11,201	0	11,693	Entrenar el modelo E3
eccv_cis_val	10	2,573	979	1,691	Elegir la época del modelo E3
eccv_cis_test	10	12,858	1,151	12,263	E3: cámaras vistas
eccv_trans_test	9	18,140	1,732	17,125	E3: cámaras nuevas

3.2. Modelo y entrenamiento (E1)

Ajustamos YOLOv8s [7] desde pesos preentrenados en COCO, que ya incluyen animales genéricos. Elegimos la versión "s" como equilibrio entre capacidad y tiempo: con 113,642 imágenes por época, "n" pierde precisión y "m" duplica el costo de entrenamiento. Entrenamos tres modelos con la misma configuración: el **principal** (full_train, E1, E2, E4 y E5), el **modelo E3** (solo eccv_train) y una **ablación** con menos fotos vacías. Las decisiones documentadas son:

- **Resolución de entrada:** 640 px, la nativa de los pesos preentrenados. Las fotos originales (hasta 2048 × 1494) se reducen al cargarlas; dos tercios de las cajas ocupan menos del 4.6 % de la imagen, así que la resolución es una limitación conocida para los animales pequeños.
- **Aumento de datos:** mosaic (apagado en las últimas 10 épocas), variación HSV (0.015 / 0.7 / 0.4) para simular cambios de luz, volteo horizontal 0.5, traslación 0.1 y escala 0.5. Sin rotación, perspectiva, volteo vertical ni mixup, porque las cámaras están fijas y niveladas.
- **Imágenes vacías como fondo:** cada foto vacía tiene un archivo de etiqueta vacío, de modo que cualquier detección en ella es un falso positivo. Usamos las 94,856 vacías de full_train (83 % del entrenamiento) porque el entregable pide manejarlas como clase de fondo, porque el desbalance es la dificultad central del problema y porque E5 debe reportar la tasa de vacías. La ablación *full_bg18786* repite el entrenamiento con 18,786 vacías al azar (50 % de fondo).
- **Optimización:** 40 épocas como máximo con parada temprana (paciencia 10), batch automático (~45), semilla 0 y optimizador elegido por Ultralytics (MuSGD, lr = 0.01 para el principal y la ablación; AdamW, lr ≈ 0.0005 para el modelo E3, que tiene menos iteraciones). Se guarda el modelo con mejor mAP en validación.

Entrenamos en Colab Pro (GPU NVIDIA L4). Encontramos un problema que afecta la reproducibilidad: Ultralytics trata como "corruptos" los JPEG sin marcador de fin de imagen (el 75 % de CCT) y los vuelve a guardar. En el primer intento recomprimió 63,057 fotos, alteró sus píxeles, llenó el disco y dejó 25,838 imágenes fuera del entrenamiento. Restauramos los originales y agregamos los 2 bytes del marcador, lo que no cambia la imagen decodificada (src/data/fix_jpegs.py).

3.3. Evaluación

Reportamos mAP@0.5 y mAP@0.5:0.95 por especie con Ultralytics. Para E3 evaluamos el modelo E3 y también el principal en eccv_cis_test y eccv_trans_test, y medimos la fracción de fotos vacías con al menos una detección (falsa alarma) con umbral 0.25. Estas falsas alarmas, y el recall y la precisión de las Tablas 2 y 4, se calculan sobre todas las cajas juntas con umbral 0.25 (IoU ≥ 0.5, misma especie); el recall de la Tabla 3 es el de Ultralytics, que promedia por especie en el umbral de máximo F1, por eso las cifras no coinciden. Para ver si el backbone atiende al animal o al fondo aplicamos EigenCAM [8] en la capa SPPF (fin del backbone) a 500 fotos por partición, y medimos la *concentración*: la fracción de activación dentro de las cajas dividida por la fracción de imagen que ocupan (1 equivale al azar).

Para E4 dividimos las cajas de full_test por tamaño relativo (área de la caja sobre área de la imagen) en tres grupos con los terciles de full_train (1.91 % y 4.59 %) y calculamos el mAP con pycocotools restringido a cada rango. No usamos los grupos small/medium/large de COCO que reporta Ultralytics porque se definen en píxeles absolutos (32^2 y 96^2) y con fotos de 2048 px casi

todo cae en "large". CCT no trae etiquetas de oclusión, así que anotamos a mano 298 cajas de full_test (20 por especie, al azar) en tres niveles: baja (animal casi completo), media (~25-50 % oculto) y alta (se ve menos de la mitad), contando como oculto lo tapado por vegetación, otros objetos o el borde de la foto. Como un falso positivo no se puede asignar a un nivel de oclusión, para este análisis reportamos el recall (IoU ≥ 0.5, misma especie, confianza ≥ 0.25).

3.4. Módulo de monitoreo ecológico (E5)

El módulo (src/monitoring) aplica reglas fijas a las detecciones de todas las fotos de una cámara, vacías incluidas, como en un despliegue real:

- **R0, corrección de hora:** si una foto infrarroja está registrada a las 12:xx, se reasigna a las 0:xx (una foto infrarroja no puede ser de mediodía).
- **R1, umbral:** solo cuentan las detecciones con confianza ≥ 0.25. **R2, conteo:** los individuos de una foto son sus cajas de esa especie.
- **R3, eventos:** cada secuencia de disparo (normalmente 3 fotos) se agrega en un evento por especie, confirmado si la especie aparece en al menos 2 fotos o con confianza ≥ 0.5; los individuos del evento son el máximo por foto.

El reporte entrega el conteo de detecciones y eventos, las especies distintas, la tasa de fotos vacías frente a fotos con fauna y la actividad por especie en cuatro periodos (madrugada 0-6 h, mañana 6-12 h, tarde 12-18 h y noche 18-24 h) y por mes. La demostración usa la cámara 46, que pertenece a full_test y no fue vista en entrenamiento.

4. Resultados

4.1. Entrenamiento y ablación de las imágenes vacías (E1)

El modelo principal completó 40 épocas en 10.9 h; la mejor fue la 30 (mAP@0.5 = 0.556 en full_val, 19 cámaras no vistas). Después de apagar mosaic la pérdida de entrenamiento siguió bajando mientras la de validación subía levemente, un inicio de sobreajuste. La Tabla 2 compara el modelo principal con la ablación en full_test.

Tabla 2: Ablación de imágenes vacías en full_test (43,292 fotos, 19,892 vacías). Recall, precisión y falsas alarmas con umbral 0.25, a nivel de caja.

Modelo	Vacías en train	Tiempo	mAP@0.5	mAP@.5:.95	Recall	Precisión	Vacías con falsa alarma
Principal	94,856 (83 %)	10.9 h	0.481	0.310	0.624	0.700	2.4 %
Ablación	18,786 (50 %)	3.9 h	0.485	0.311	0.660	0.649	5.4 %

Las vacías no cambian el mAP, porque el mAP se calcula sobre las cajas reales y castiga poco las detecciones en fotos sin animales. Lo que cambian es la calibración: con más fondo, el modelo da menos confianza a lo que no es un animal, y con el mismo umbral produce menos de la mitad de falsas alarmas (unas 600 fotos vacías menos que revisar en este test), a cambio de encontrar menos animales. Comparados con la misma tasa de falsas alarmas (~2.4 %), el principal recupera algo más (0.624 frente a ~0.607), así que las vacías aportan una mejora real aunque pequeña frente a su costo (2.8 veces más tiempo de entrenamiento). Mantenemos el modelo principal.

4.2. Evaluación por especie y efecto del desbalance (E2)

Tabla 3: Modelo principal en full_test (39 cámaras no vistas): AP por especie, ordenado por cajas de entrenamiento.

Especie	Cajas train	Cajas test	AP@0.5	AP@.5:.95	Recall
rabbit	3,255	753	0.49	0.24	0.56
opossum	2,772	4,972	0.87	0.43	0.79
deer	2,532	3,017	0.66	0.48	0.52
coyote	2,273	2,293	0.62	0.43	0.68
raccoon	1,736	4,832	0.81	0.52	0.69
bird	1,344	1,291	0.16	0.09	0.15
cat	1,178	1,392	0.29	0.18	0.37
squirrel	1,079	796	0.27	0.15	0.18
bobcat	1,021	2,844	0.68	0.45	0.58
dog	910	1,010	0.71	0.48	0.62
rodent	842	424	0.55	0.35	0.50
skunk	429	469	0.74	0.53	0.77
fox	409	428	0.35	0.31	0.36
lizard	32	123	0.00	0.00	0.00
badger	5	18	0.02	0.01	0.00
Todas	19,861	24,662	**0.481**	**0.310**	0.451

El desbalance se nota en los extremos: las especies con muy pocos ejemplos no se detectan (lagartija, 32 cajas: AP 0; tejón, 5 cajas: 0.02; zorro, 409: 0.35), mientras que zarigüeya y mapache, con miles de cajas, superan 0.8. Sin embargo, la correlación de Spearman entre cajas de entrenamiento y AP@0.5 es solo 0.42 (Figura 2): el ave tiene 1,344 cajas y AP 0.16, y la ardilla 1,079 y 0.27, porque son animales pequeños y difíciles de ver. El tamaño del animal pesa tanto como la cantidad de ejemplos. El puma no tiene cajas en full_test y no se evalúa.

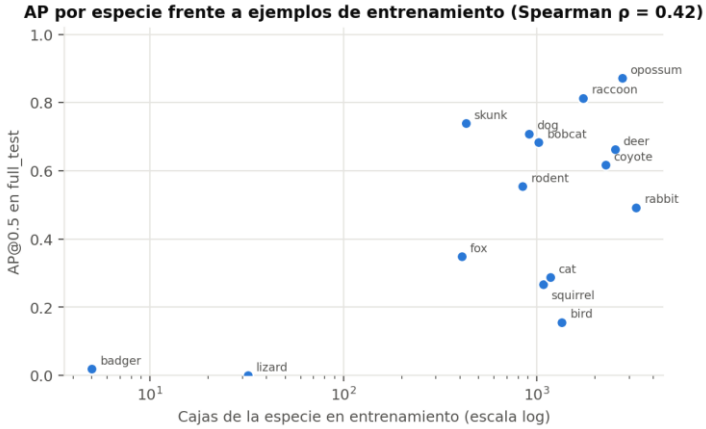


Figura 2: AP@0.5 por especie en full_test frente al número de cajas de entrenamiento (escala logarítmica).

4.3. Generalización entre cámaras (E3)

Con la partición oficial, el modelo E3 pasa de mAP@0.5 = 0.746 en cámaras vistas a 0.420 en cámaras nuevas, una brecha de 0.326 (−44 %); en mAP@0.5:0.95 cae de 0.485 a 0.261 (Figura 3). El modelo principal, entrenado con 76 cámaras en lugar de 10, tiene casi la misma brecha (0.766 → 0.453): más cámaras mejoran poco las cámaras nuevas (+0.033). Entre las especies con suficientes cajas en ambos conjuntos, las que más caen son ave (0.62 → 0.05), gato (0.91 → 0.28), ardilla (0.69 → 0.29) y coyote (0.91 → 0.57); el roedor cae de 0.79 a 0.00, pero solo tiene 22 cajas en trans, y el zorro (2 y 1 cajas) no se puede comparar. Zarigüeya y mapache, nocturnas y con muchos ejemplos, casi no caen. Parte de la brecha se debe también a que las cámaras trans tienen otra mezcla de especies.

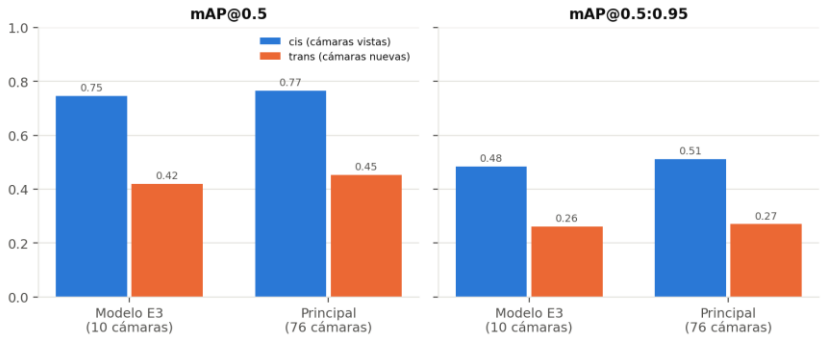


Figura 3: mAP en cámaras vistas (cis) y nuevas (trans) para el modelo E3 y el modelo principal.

La evidencia más clara de que el modelo E3 aprende el fondo está en las fotos vacías: genera falsas alarmas en el 16 % de las vacías de sus cámaras y en el 38 % de las de cámaras nuevas, 2.3 veces más. No aprendió qué es un fondo en general, sino cómo son los fondos de sus 10 cámaras (eccv_train no trae vacías). El modelo principal, que vio 94,856 vacías de 76 cámaras, baja a 4-6 % y casi no cambia entre cis y trans.

EigenCAM apunta en la misma dirección (Figura 4): en la capa SPPF la activación dominante cubre rocas, vegetación y suelo, y su concentración en las cajas es 0.94 en cis y 0.94 en trans para el modelo E3 (≈ azar); el punto de máxima activación cae dentro del animal solo en el 5-9 % de las fotos. En trans, los animales que sí se detectan reciben menos activación relativa que en cis (0.71 frente a 0.88). Hay que leer este resultado con cautela: la métrica promedio no separa cis de trans, y EigenCAM resume la estructura dominante de la imagen, que con animales que ocupan ~5 % de la foto es casi siempre la escena. Por eso lo usamos como evidencia cualitativa junto con las falsas alarmas, que son una medida directa.

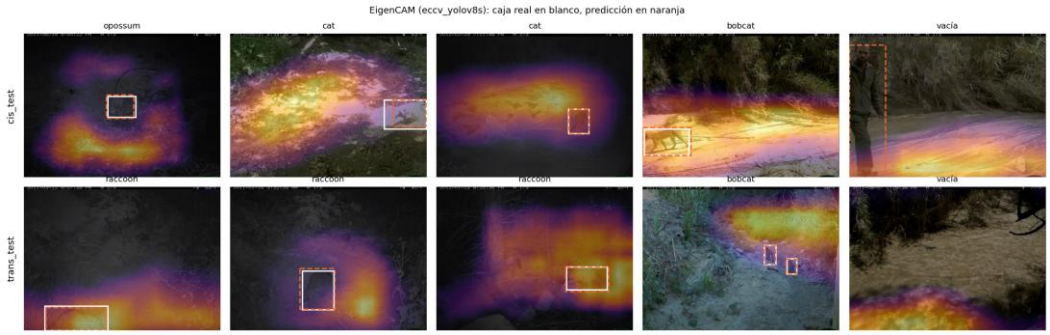


Figura 4: EigenCAM del modelo E3 (capa SPPF) en cámaras vistas (arriba) y nuevas (abajo); caja real en blanco y predicción en naranja. La última columna son fotos vacías (arriba, una persona etiquetada como vacía).

4.4. Robustez ante tamaño y oclusión (E4)

Las cajas pequeñas (< 1.91 % del área) tienen mAP@0.5 = 0.37 frente a 0.51 de las medianas y 0.49 de las grandes; en mAP@0.5:0.95 la caída es mayor (0.20 frente a 0.36), porque lo que más sufre es la localización precisa (Tabla 4). Las grandes no superan a las medianas porque suelen ser animales pegados a la cámara y cortados por el borde. La oclusión degrada mucho más: de las 298 cajas anotadas, 200 tienen oclusión baja, 54 media y 44 alta, y el recall cae de 0.63 a 0.24 y 0.16 (Tabla 4). Con oclusión, además, el modelo confunde más la especie: localiza al 34 % de los animales muy ocultos pero solo acierta la especie en el 16 %.

Tabla 4: Robustez del modelo principal en full_test. Arriba, mAP por tamaño relativo de la caja (terciles de full_train). Abajo, recall por nivel de oclusión en 298 cajas anotadas a mano ("sin especie": localizado aunque con la especie equivocada; última columna: ablación).

Grupo	Cajas	mAP@0.5 / Recall	mAP@.5:.95 / Sin especie	Ablación
Tamaño pequeño (< 1.91 %)	8,242	0.37	0.20	—
Tamaño mediano (1.91-4.59 %)	8,395	0.51	0.33	—
Tamaño grande (> 4.59 %)	8,025	0.49	0.36	—
Oclusión baja	200	0.63	0.74	0.66
Oclusión media	54	0.24	0.41	0.31
Oclusión alta	44	0.16	0.34	0.18

Esta comparación tiene un sesgo: la oclusión no se reparte igual entre especies. Lagartija, roedor, zorro y ardilla concentran muchas de las cajas con oclusión media o alta, y son especies que ya eran difíciles, mientras que conejo, coyote y perro casi no tienen. Parte de la caída se debe a la especie y no solo a la oclusión, y con 44-54 cajas por nivel la incertidumbre es de unos ±0.1. La Figura 5 muestra ejemplos de cada nivel.

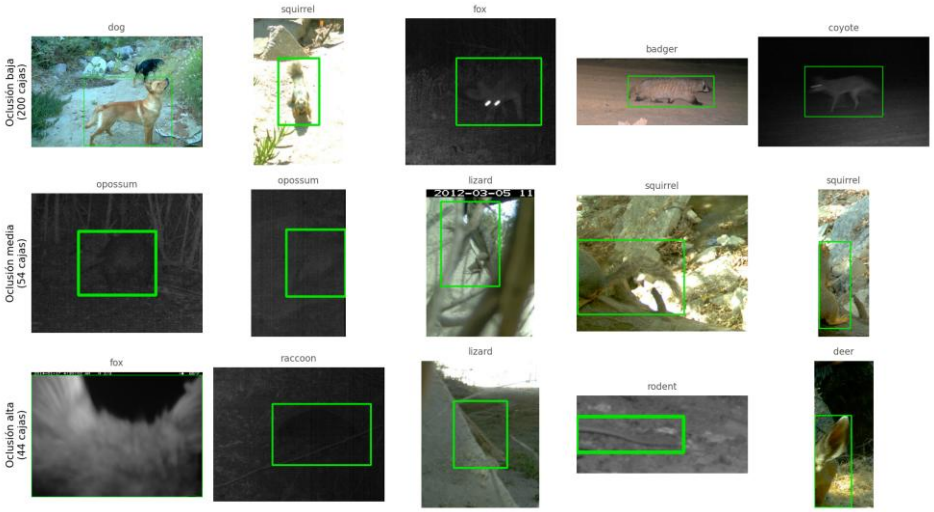


Figura 5: Ejemplos de la anotación manual de oclusión en full_test: baja (arriba), media y alta (abajo). En las fotos nocturnas el nivel se asignó según la parte del animal que se distingue.

4.5. Monitoreo en una cámara no vista (E5)

La cámara 46 tiene 6,035 fotos en 2,124 secuencias, de mayo de 2011 a junio de 2012. La regla R0 corrigió la hora de 444 fotos. El modelo reporta 44.3 % de fotos vacías cuando en realidad son 11.6 %: pierde animales, coherente con su recall. A nivel de evento detecta 1,457 frente a 1,873 reales, con 1,102 correctos (precisión 0.76, recall 0.59). La Tabla 5 muestra el reporte por especie y periodo, y la Figura 6 la actividad por hora.

Tabla 5: Reporte de actividad de la cámara 46: eventos detectados por el modelo (reales entre paréntesis), por especie y periodo del día.

Especie	Madrugada	Mañana	Tarde	Noche	Total
opossum	240 (278)	6 (11)	0 (0)	146 (158)	392 (447)
raccoon	95 (284)	2 (1)	0 (1)	32 (219)	129 (505)
bobcat	116 (121)	87 (82)	28 (36)	92 (96)	323 (335)
coyote	53 (42)	111 (169)	28 (40)	43 (41)	235 (292)
rabbit	24 (39)	82 (116)	1 (3)	21 (31)	128 (189)
skunk	77 (20)	6 (8)	0 (0)	72 (9)	155 (37)
cat	17 (3)	11 (3)	11 (0)	10 (1)	49 (7)

El reporte recupera los patrones de actividad: zarigüeya y mapache nocturnos, coyote y conejo con pico en la mañana, lince repartido en el día. Los errores son sistemáticos: el mapache queda muy subcontado (129 frente a 505 eventos) y el zorrillo sobrecontado (155 frente a 37), lo que sugiere una confusión entre ambos de noche en infrarrojo; además aparecen zorro y venado, que no existen en la cámara. Un especialista tendría que revisar esos eventos antes de usar el reporte.

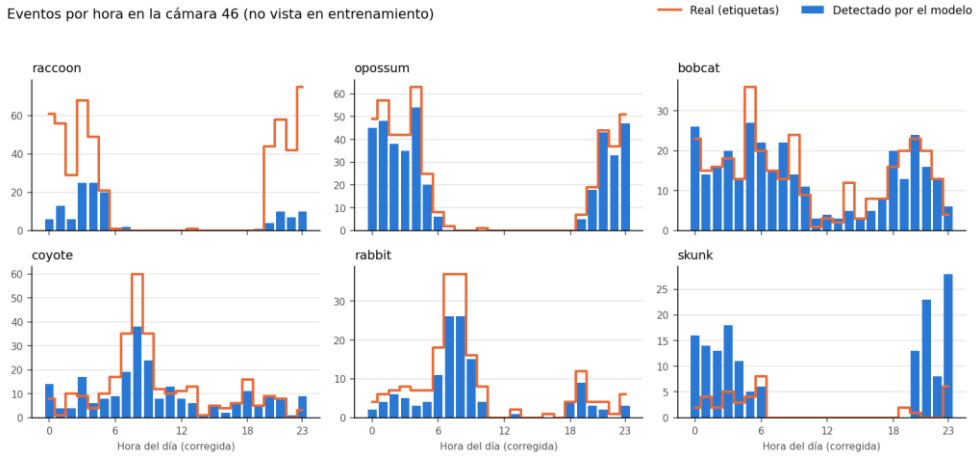


Figura 6: Eventos por hora en la cámara 46: detectados por el modelo (barras) y reales (línea).

5. Discusión

5.1. Uso responsable de IA en conservación (E6)

En conservación, los dos errores del detector no cuestan lo mismo. Un **falso negativo** sobre una especie amenazada puede borrarla de un inventario y dejarla fuera de las decisiones de manejo; es un error silencioso, porque nadie revisa las fotos que el modelo marcó como vacías. Un **falso positivo** cuesta tiempo de un especialista, pero se corrige al revisar. Nuestros resultados muestran que este compromiso se controla con el umbral de confianza (Figura 7): en full_test, bajar el umbral de 0.25 a 0.05 sube el recall de 0.62 a 0.70 a cambio de triplicar las fotos vacías con falsa alarma (de 2.4 % a 7.2 %); subirlo a 0.70 deja el recall en 0.42. Para una especie amenazada conviene un umbral bajo y revisión humana de todo lo detectado; para un inventario masivo de especies comunes, un umbral más alto. El umbral debería fijarse con el equipo que usará el reporte, no por defecto, y las especies raras deberían revisarse siempre a mano: el modelo no detecta ninguna lagartija y casi ningún tejón.

El segundo riesgo es desplegar el modelo en otra región sin validarlo. En nuestras propias cámaras de Estados Unidos, el mAP cae 44 % al pasar a cámaras nuevas, y un modelo entrenado sin fondo inventa animales en el 38 % de las fotos vacías. En la Amazonía o los Andes peruanos cambiarían la vegetación, la luz y, sobre todo, las especies, que el modelo no conoce y clasificaría como la especie más parecida de su lista. Antes de usarlo habría que anotar una muestra local, medir el desempeño por especie en esas cámaras y reentrenar con datos de la región. El reporte de la cámara 46 lo ilustra: reporta especies que no existen en el lugar y subcuenta la más frecuente, errores que solo se ven al comparar con etiquetas locales.

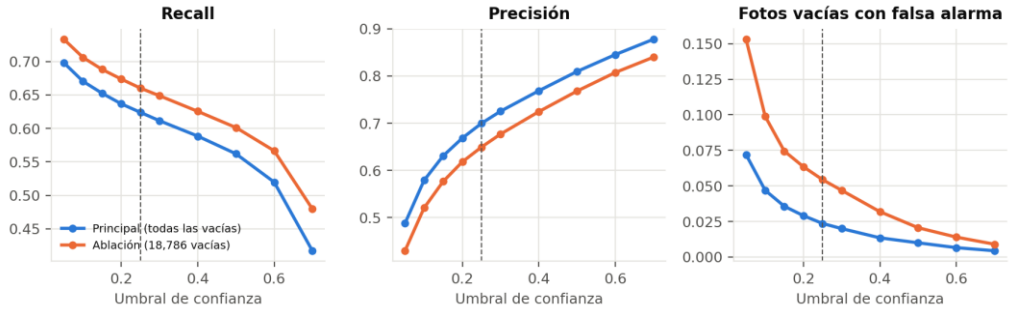


Figura 7: Efecto del umbral de confianza en full_test: recall, precisión y fotos vacías con falsa alarma (la línea vertical marca 0.25).

5.2. Limitaciones

- La resolución de 640 px limita la detección de animales pequeños; no pudimos probar 1024 px por el costo de GPU (unas 2.5 veces más cómputo).
- La oclusión la anotó una sola persona sobre 298 cajas, sin medir acuerdo entre anotadores, y la muestra mezcla oclusión con especie.
- Las etiquetas de CCT son por secuencia y las fotos con personas o vehículos están marcadas como vacías, así que parte de las "falsas alarmas" son objetos reales y parte de los "falsos negativos" son fotos sin animal visible.
- El modelo E3 y el principal usaron optimizadores distintos (AdamW y MuSGD, elegidos automáticamente por Ultralytics), lo que añade una diferencia a la comparación entre ambos además del número de cámaras.
- EigenCAM resume la componente dominante de las activaciones y no es específico de la clase; su lectura es cualitativa.
- Las horas de los metadatos tienen errores; R0 corrige el error AM/PM en fotos infrarrojas, pero no los relojes mal configurados.
- No hicimos la extensión opcional con un segundo dataset de otra región de LILA BC, que permitiría medir la generalización geográfica real y no solo entre cámaras de la misma zona.

6. Conclusiones

- YOLOv8s entrenado con todas las fotos vacías como fondo alcanza mAP@0.5 = 0.481 en 39 cámaras nunca vistas; las vacías no suben el mAP, pero reducen a menos de la mitad las falsas alarmas en fotos sin animales.
- El desbalance anula a las especies con decenas de ejemplos, pero el tamaño del animal explica tanto como la cantidad de datos (Spearman 0.42).
- El modelo pierde 44 % de mAP en cámaras nuevas y, si se entrena sin vacías, inventa animales en el 38 % de los fondos desconocidos; más cámaras de entrenamiento reducen las falsas alarmas pero casi no la brecha de mAP.
- La oclusión es el factor que más degrada la detección (recall de 0.63 a 0.16), por encima del tamaño.
- El módulo de reglas produce un reporte útil de actividad en una cámara nueva, pero sus conteos por especie necesitan revisión humana antes de usarse en decisiones de manejo.

Como trabajo futuro proponemos entrenar a mayor resolución o con recortes del animal, usar pérdidas o muestreo que compensen las especies raras, y ajustar el modelo con una muestra anotada de cada nueva región antes de desplegarlo.

7. Material complementario

El código, los notebooks, las tablas de resultados y las anotaciones de oclusión están en https://github.com/Roxana-Vargas/redes-parcial-p3. Todo el flujo se ejecuta con bash scripts/run_all.sh (descrito en el README); los pesos entrenados están publicados en el

repositorio.

Referencias

[1] Beery, S., Van Horn, G., & Perona, P. (2018). Recognition in Terra Incognita. *European Conference on Computer Vision (ECCV)*, 456–473.

[2] Norouzzadeh, M. S., Nguyen, A., Kosmala, M., Swanson, A., Palmer, M. S., Packer, C., & Clune, J. (2018). Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. *Proceedings of the National Academy of Sciences*, 115(25), E5716–E5725.

[3] Beery, S., Morris, D., & Yang, S. (2019). Efficient pipeline for camera trap image review. *arXiv:1907.06772*.

[4] Tabak, M. A., Norouzzadeh, M. S., Wolfson, D. W., et al. (2019). Machine learning to classify animal species in camera trap images: Applications in ecology. *Methods in Ecology and Evolution*, 10(4), 585–590.

[5] Schneider, S., Taylor, G. W., & Kremer, S. (2018). Deep learning object detection methods for ecological camera trap data. *15th Conference on Computer and Robot Vision (CRV)*, 321–328.

[6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[7] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[8] Muhammad, M. B., & Yeasin, M. (2020). Eigen-CAM: Class activation map using principal components. *International Joint Conference on Neural Networks (IJCNN)*, 1–7.

[9] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *IEEE International Conference on Computer Vision (ICCV)*, 618–626.

[10] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740–755.